Claude Code for Academic Researchers

A Methods Workshop in AI-Assisted Research

Brady Allardice

UPF & IBEI

About Me

Brady Allardice
PhD Candidate, Political & Social Sciences
UPF / IBEI, Barcelona

Research: Political economy of technological change

- Automation, AI, and political behavior
- LLM-based text classification pipelines

Background:

- M.Res. Political Science (UPF)
- M.Sc. Economics (Barcelona School of Economics)
- Former data scientist

Why this course:

- Daily user of Claude Code for academic research
- Use it across the full pipeline: data cleaning, analysis, robustness checks, code auditing, replication, writing, reviewing
- These tools can dramatically expand what a researcher can accomplish, if used with the right workflows

brady.allardice@upf.edu
github.com/bradyall

Introductions

1. Who are you? (name, department, research area)
2. What does your daily workflow look like? (tools, languages, data)
3. How do you currently use AI, if at all?
4. What do you hope to learn from this course?

Course Roadmap

Session	Modules	Date
1	1. What This Is & What You Can Do With It 2. How Claude Code Works	Today
2	3. Automating Research Tasks 4. Robustness and Replication	Thu Apr 9, 11am–1pm
3	5. Code Auditing and Review 6. Version Control with Git	Fri Apr 17, 11am–1pm
4	7. Developing Expert Judgment 8. The Anthropic API 9. Capstone Project	Fri Apr 24, 11am–1pm

4 sessions $\times$ 2 hours | 60–70% hands-on | Pair programming throughout

Possible 5th session: Thu Apr 30, 11am–1pm, depending on participants' needs

Core Framework

The Central Idea

You are the principal investigator. Claude Code is the research assistant.
You direct the work, make analytical decisions, and verify every result.
Claude Code handles the mechanical execution.

The question is never "can Claude Code do this." It almost always can.

The question is: **what is the most productive and reliable way to get this done?**

By the end of this course, you will be able to:

- Automate data cleaning, robustness checks, code auditing, and documentation
- Choose the right execution mode for each task
- Verify output against known properties of your data
- Recognize when Claude Code is failing and intervene effectively

Module 1

What This Is and What You Can Do With It

Demo + Exercise

Live Demo: The Data

Dataset 1: County Presidential Election Returns

- Source: MIT Election Data + Science Lab
- County-level vote totals by candidate and party, 2000–2024
- $\sim$94,000 rows, one row per county $\times$ year $\times$ candidate
- Variables: FIPS codes, candidate names, party, vote counts, vote mode

Dataset 2: IPUMS American Community Survey Microdata

- Source: IPUMS USA (University of Minnesota)
- Person-level survey records with demographic, economic, and geographic variables
- $\sim$15 million rows per year, one row per person
- Variables: age, education, employment, income, race, poverty status, county FIPS

Live Demo: The Task

Goal: Build a county-year panel joining election results with demographics, and visualize the outputs.

This requires:

1. Reshape election data to one row per county-year with vote shares
2. Aggregate 15M person-level records to county-level means using survey weights
3. Harmonize FIPS codes across datasets (different formats)
4. Merge on county $\times$ year and validate the join
5. Create visualizations of trends over time

In Stata or R, this takes 30–45 minutes of careful coding.

What Just Happened

It wrote the code:

- Read both files
- Identified the join key
- Handled cleaning steps
- Wrote a complete script

It also:

- Understood the data structure (person-level vs. county-level, survey weights, FIPS formats)
- Ran the code and diagnosed errors
- Provided summaries of the output (row counts, variable descriptions, merge diagnostics)

But it also made a decision about our data without asking.

Both of these things are true:

1. It took 90 seconds instead of 45 minutes
2. It made analytical choices autonomously

The RA Analogy

How to Think About Claude Code

Claude Code is like a research assistant who is:

- Extremely fast
- Has read a lot
- Will **never** say "I don't know how to do that"

Instead, it will just do it wrong and hand it in.

You have all managed RAs. You know how valuable, and how dangerous, that profile is.

This course teaches you to capture the value while managing the risk.

The Coding Paradigm Spectrum: Manual Coding

Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding

You write every line. Full control, no assistance.

This is what you do now in Stata or R.

```python
def load_tasks_to_dwas(tasks_to_dwas_file: str) -> pd.DataFrame:
    """
    Load Tasks-to-DWAs mapping file.

    Returns DataFrame with columns: O*NET-SOC Code, Task ID, DWA ID, DWA Title
    """
    if not os.path.exists(tasks_to_dwas_file):
        raise FileNotFoundError(f"Tasks-to-DWAs file not found: {tasks_to_dwas_file}")

    df = pd.read_excel(tasks_to_dwas_file)

    required_cols = ['O*NET-SOC Code', 'Task ID', 'DWA ID', 'DWA Title']
    missing = [c for c in required_cols if c not in df.columns]
    if missing:
        raise ValueError(f"Missing columns in Tasks-to-DWAs file: {missing}. Available: {list(df.columns)}")

    logger.info(f"Loaded {len(df)} task-DWA mappings ({df['Task ID'].nunique()} unique tasks, "
                f"{df['DWA ID'].nunique()} unique DWAs)")
    return df


def load_dwa_reference(dwa_reference_file: str) -> pd.DataFrame:
    """
    Load DWA Reference file with IWA (Intermediate Work Activity) context.
```

The Coding Paradigm Spectrum: Autocomplete / Copilot

Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding

```typescript
// create a class in TypeScript to represent a student that has a name, an id, and a list of courses
// the class should have method to add and remove a course to the list of courses

class Student {
  name: string;
  id: number;
  courses: string[];

  constructor(name: string, id: number, courses: string[]) {
    this.name = name;
    this.id = id;
    this.courses = courses;
  }

  addCourse(course: string) {
    this.courses.push(course);
  }

  removeCourse(course: string) {
    this.courses = this.courses.filter((c) => c !== course);
  }
}
```

AI suggests the next line as you type.

You accept or reject. The AI assists; you drive.

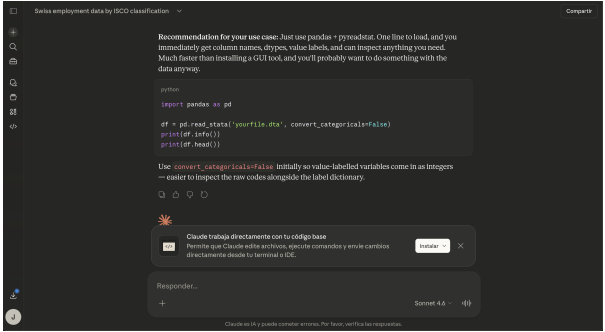
The Coding Paradigm Spectrum: Web Chat

Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding



You describe a task; AI gives you code to copy-paste.

The AI cannot see your project.

The Coding Paradigm Spectrum: Agentic Coding

Manual Coding

Autocomplete / Copilot

Web Chat

Agentic Coding

AI reads, writes, and runs code **in your project**.

So 69 fuzzy + 241 low-score token = **310 borderline matches** the LLM confirmed but we're not fully confident in. Let me flag those as `needs_review=True`:

Read `improve_shp_firm_matching.py` (lines 1639-1663)

Edit `improve_shp_firm_matching.py`

Added 10 lines

```
# --- Flag borderline matches as needs_review even if LLM confirmed ---
# fuzzy_char and low-score token matches are inherently less reliable
borderline_mask = (
    (df["match_method"] == "fuzzy_char") |
    ((df["match_method"].isin(["token_exact", "token_relaxed"])) & (df["match_score"]
    <= 0.50))
)
n_borderline_flagged = ((borderline_mask) & (~df["needs_review"])).sum()
df.loc[borderline_mask, "needs_review"] = True
logger.info(f"  Borderline flagged: {n_borderline_flagged} matches set to
needs_review=True")
```

Bash Re-compile with borderline flagging

```
IN   python3 improve_shp_firm_matching.py compile --output-dir Data/shp_matching/ 2>&1

OUT  2026-03-05 21:53:32 [INFO] Loaded 11780 raw matches from matched_names_raw.csv
     2026-03-05 21:53:32 [INFO]   LLM rejected: removed 532 matches
     2026-03-05 21:53:32 [INFO]   Manual verdicts: removed 4, confirmed 10
```

Final: 11,244 matched — 10,842 confirmed, 402 flagged `needs_review=True`.
The 402 flagged entries are:

Why Agentic Coding Is Different

Paradigm	Pros	Cons
Manual Coding	Full control	Slow, everything by hand
Autocomplete / Copilot	You control logic and structure	Still line-by-line
Web Chat	Can handle complex tasks	Copy-paste; AI misses context
Agentic Coding	**Fastest; works on your project**	**Can act autonomously**

The Key Insight

Agentic coding is the first paradigm where the tool **interacts with your project directly and autonomously**. That is what makes it so productive, and why learning to use it well matters.

These principles apply to Claude Code, Cursor, Codex, Windsurf, or whatever comes next.

What Claude Code Can Do for Academic Research

Data Work

- Clean, reshape, and merge datasets
- Construct variables from raw data
- Build panel datasets across sources
- Handle missing data, recoding, harmonization

Analysis

- Exploratory data analysis and summary statistics
- Generate robustness checks and specification variants
- Create publication-quality visualizations
- Synthesize results across specifications

Writing and Review

- Draft methods sections, data descriptions, results summaries
- Audit existing scripts for silent errors
- Review code against your methods section
- Write documentation, READMEs, replication packages

Research Infrastructure

- Process text corpora at scale (NLP, classification)
- Web scraping and API data collection
- Crosswalk and linkage construction
- Refactor and optimize slow pipelines

A Tool Built for Software Engineers

Claude Code was designed for professional developers. Their world is different from ours.

Software Engineering

- "Works" = **runs correctly**. Clear success criteria: tests pass, feature ships, users are happy
- Code is the product
- Errors are caught by automated tests
- Speed and iteration are paramount

Academic Research

- "Works" = **???**. Code that runs without errors can produce completely wrong results. Output that looks plausible can be based on the wrong sample, wrong specification, or wrong variable
- Code is the *instrument*; the paper is the product
- Errors are caught by *you*, or by Reviewer 2
- Correctness matters more than speed

The defaults, instincts, and shortcuts built into these tools reflect engineering priorities.
Using them well for research means overriding some of those defaults.

Do You Own It?

Pick any result your code produced. Without looking at the code:
what got dropped, what got merged, and why?

If you can't answer that in 30 seconds, you don't own it.

Before you delegate:

- What would wrong look like? Can I describe a specific failure mode before I run this?

After you delegate:

- Can I describe what this code does without reading it?
- Can I state what decisions shaped the output and why?

You didn't write it. That's fine.
But every decision in it is yours. **Can you justify it as if you chose it yourself?**

Why Verification Matters
What happens when you skip verification:

1. **Silent observation loss:** A merge silently drops 300 observations. The code looks clean. The error is invisible without checking row counts.
2. **Invalid transformation:** A log transformation applied to a variable containing zeros. No error raised, wrong results.
3. **Incorrect panel operations:** A lagged variable computed without sorting by time within panels.

These are the kinds of errors that end up in published work.

The Control Principle

You Are the PI. Claude Code Is the RA.

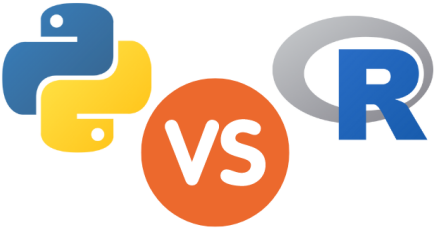
The question is not "can Claude Code do this" but "what is the best way to get this done reliably."

If you cannot verify a result, you should not delegate the task.

That is the standard we hold ourselves to today.

Why Python

- Claude Code is substantially better at Python than Stata or R
- The principles (task decomposition, verification, control) transfer to any language
- Python is increasingly where the tools and ecosystem are for computational research

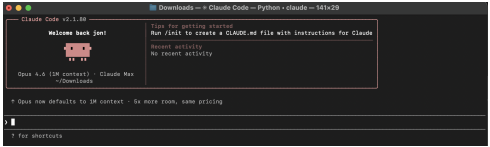


Where Does Claude Code Live?

Claude Code runs **inside your terminal**, not in a browser, not in an app.

1. You open a terminal
2. Navigate to your project: `cd my_project/`
3. Type `claude` and press Enter
4. You're now talking to Claude Code

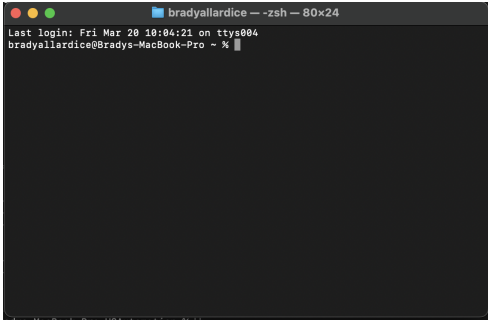
It can see your files, read your data, write scripts, and run them, all without copying and pasting.



This is what Claude Code looks like when you launch it.

What Is the Terminal?

The terminal is a text-based interface to your computer. Instead of clicking, you type commands.



Claude Code writing and running code in the terminal.

Essential commands:

```
# Where am I?
pwd

# What's in this folder?
ls

# Move into a folder
cd my_project/

# Run a Python script
python my_script.py

# Launch Claude Code
claude
```

Why VS Code?

VS Code is a free code editor with a built-in terminal.

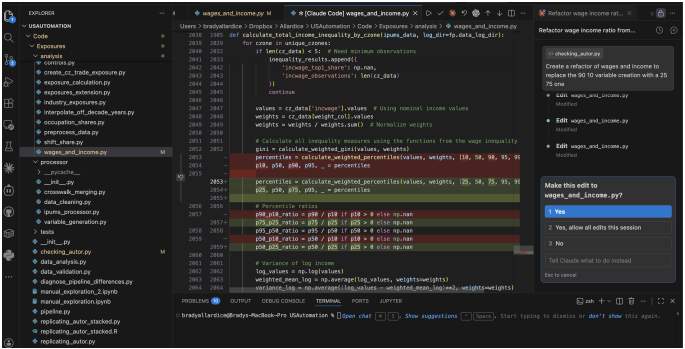
Everything in one window:

- **Left:** file explorer
- **Center:** your code
- **Right:** Claude Code

You see what Claude Code writes *as it writes it*.

Today we use three features:

1. Open a folder
2. View files in the editor
3. Talk to Claude Code



Coming from RStudio? ▸ Configure VS Code to look like RStudio

Python Files: `.py` vs. `.ipynb`

Notebooks (`.ipynb`)

```python
df_ipums_pre_exposure = pd.read_pickle(f'/Users/bradyallardice/Desktop/PhD/Projects/USAutomation/Data/Merge/IPUMS/processed_ipums_1040.pkl')
df_ipums_pre_exposure.groupby(['occ1950rj','czone'])['pervet_by_afactor'].sum().reset_index()
```

[39] Python

	occ1950rj	czone	pervet_by_afactor
0	2.0	3700.0	62.676506
1	2.0	3800.0	37.335463
2	2.0	4200.0	9.049831
3	2.0	4301.0	12.407893
4	2.0	4702.0	12.759623
...	...	...	...
68909	970.0	39206.0	238.110368
68910	970.0	39301.0	83.274260
68911	970.0	39302.0	1712.524176
68912	970.0	39303.0	1064.610787
68913	970.0	39400.0	25848.531526

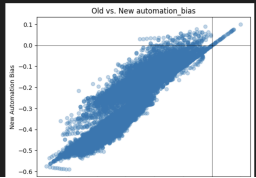
68914 rows × 3 columns

Visualizing Automation Exposure Changes

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.scatter(df_final['automation_bias_per_worker'], df_final['automation_bias_per_worker_ffilled'], alpha=0.3)
ax.set_xlabel('Old Automation Bias')
ax.set_ylabel('New Automation Bias')
ax.axhline(0,color='k',linewidth=.5); ax.axvline(0,color='k',linewidth=.5)
ax.set_title('Old vs. New automation_bias')
plt.show()
```

[0] Python



Scripts (`.py`)

Code > Exposures > checking_autor.py > ...

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr, spearmanr
import os
import pickle
from pathlib import Path

# Set up plotting style
plt.style.use('ggplot')
sns.set_context("talk")

import General.filepaths as fp

def load_data_files():
    """Load all intermediate data files needed for diagnostics"""
    data_files = {}

    # Try to load cz-level data
    try:
        occ_exposure_path = f'{fp.exposures_out_dir}occ_ind_4080rj_panel_cw_t85.dta'
        data_files['occ_exposure'] = pd.read_stata(occ_exposure_path)
        data_files['occ_exposure'] = data_files['occ_exposure'][['year','occ1950rj','N
        print(f"✓ Loaded occupation exposure data from {occ_exposure_path}")
        print(f"  Shape: {data_files['occ_exposure'].shape}")
    except FileNotFoundError:
        print(f"✗ Could not find occupation exposure data at {occ_exposure_path}")

    # Try to load occupation shares data
    try:
        occ_shares_path = f'{fp.exposures_out_dir}final_occ_shares_all_years_by_cz.csv
        data_files['occ_shares'] = pd.read_csv(occ_shares_path)
        print(f"✓ Loaded occupation shares data from {occ_shares_path}")
        print(f"  Shape: {data_files['occ_shares'].shape}")
    except FileNotFoundError:
        print(f"✗ Could not find occupation shares data at {occ_shares_path}")
```

Why Scripts for This Course

Notebooks (`.ipynb`)

- Cells you run one at a time
- Great for exploration and teaching
- State depends on which cells you ran and in what order
- Hard for Claude Code to work with because it cannot "click" cells

Scripts (`.py`)

- Plain text file, runs top to bottom
- Reproducible: `python my_script.py` always does the same thing
- Claude Code can read, write, and run them directly
- **This is what we use today**

The Key Difference

Claude Code operates on files, not interactive sessions. Scripts run end-to-end and produce the same result every time, exactly what you need for reproducible research.

What Changes When You Use Scripts

You can run scripts line by line in VS Code (Shift+Enter), just like a notebook. But scripts aren't really built for that.

Their strength is that they run **end-to-end**: `python my_script.py` always does the same thing, top to bottom. That is what makes them reproducible, and what Claude Code needs.

What this means for your workflow:

- More **checkpointing**: save intermediate datasets to disk so you can inspect them
- More **logging**: print row counts, shapes, and summaries as the script runs
- More **intermediate file saves**: write CSVs between pipeline stages
- Overall **less interactive**, more structured

This feels slower at first. But it produces pipelines that are reproducible, debuggable, and that Claude Code can work with directly.

Virtual Environments

What is a virtual environment?
A self-contained folder that holds a separate copy of Python and its packages. Each project gets its own environment, so packages from one project don't interfere with another.

Why you need one:

- Without it, every project shares the same packages, and updating one project can break another
- Claude Code installs packages when it needs them. You want that isolated, not touching your system Python
- Reproducibility: you know exactly which packages your project uses

Think of it like this: a virtual environment is a clean desk for each project. You only put the tools you need on it, and cleaning up one desk does not affect the others.

Setting Up a Virtual Environment

Run these commands in your terminal, from your project folder:

```
# 1. Create the environment (once per project)
python3 -m venv AIAgents

# 2. Activate it (every time you open a new terminal)
source AIAgents/bin/activate # Mac/Linux
AIAgents\Scripts\activate # Windows

# 3. Install packages
pip install pandas numpy statsmodels

# 4. Confirm it's active (you should see (AIAgents) in your prompt)
which python
```

In VS Code: once you create the `AIAgents` folder, VS Code will detect it and ask to use it as your interpreter. Say yes. After that, every terminal it opens will activate the environment automatically.

Setup Check

Open the terminal in VS Code (or your standalone terminal) and run:

1. Verify Node.js: `node --version`
2. Verify Claude Code: `claude --version`
3. Launch Claude Code: `claude`

You should see Claude Code's interface appear in your terminal.

Try typing a simple message: `What files are in this directory?`

If you completed pre-work: help your neighbor.
If you're stuck: pair with someone who is set up. We'll troubleshoot at break.

Exercise: Your First Commands

Task 1 (low-stakes):
Here is a Python script that cleans a dataset.
Ask Claude Code to explain what it does, line by line.

Task 2 (introduces verification):
Ask Claude Code to add an assertion that the row count after cleaning matches the expected count.

Stretch:
Ask Claude Code to handle a new edge case: negative values in a column that should be positive.

Exercise Debrief

- What did it get right?
- Anything wrong or confusing?
- That is normal and expected

What's Ahead

By the end of this course, you will have:

- Built a documented data pipeline
- Generated robustness checks
- Audited code for errors
- Learned version control for agent-assisted work

You will know which parts Claude Code should handle and which require your judgment.

And you will be able to take these workflows back to your own research.